

Conception Phase

Task 1: Scalable Static AI Website Hosting Architecture using AWS and Terraform

1. Architectural Overview

This project implements a **serverless, scalable, and automated cloud architecture** to host a static AI landing page using **Amazon Web Services (AWS)** and **Terraform** as Infrastructure as Code (IaC). The objective is to design a cost-efficient and globally accessible solution that ensures high availability, performance optimization, automation, and maintainability.

The architecture follows a **CDN-backed static website pattern**, where **Amazon S3** stores the static content and **Amazon CloudFront** distributes it worldwide via edge locations. The infrastructure is fully provisioned through Terraform to guarantee reproducibility and consistent deployments.

2. Selected Services and Justification

Amazon S3 (Simple Storage Service)

Amazon S3 is used to store and host the static website files (index.html). It was selected because it provides fully managed object storage, high durability, low cost, and native support for static website hosting without requiring server management.

The S3 bucket: Stores the website files , Is configured for static website hosting , Defines *index.html* as the entry point, Uses a bucket policy to allow public read access

Public access is intentionally configured using: **aws_s3_bucket_public_access_block** , **aws_s3_bucket_policy**

This ensures controlled and transparent permission management.

Amazon CloudFront (Content Delivery Network)

CloudFront acts as a global CDN in front of the S3 bucket. It improves performance by delivering content from the nearest edge location to the user. It also provides caching, scalability, optional HTTPS support, and origin protection.

CloudFront retrieves website content from the S3 bucket (origin) and distributes it globally, reducing latency and improving user experience.

Terraform (Infrastructure as Code)

Terraform is used to automate the complete infrastructure deployment. It enables version-controlled, repeatable, and modular infrastructure management.

The project structure is organized as follows:

- **provider.tf** – AWS provider configuration
- **s3.tf** – S3 bucket and website configuration
- **cloudfront.tf** – CloudFront distribution configuration
- **variables.tf** – Parameterized configuration (region, bucket name, project name, origin ID)
- **outputs.tf** – CloudFront domain output

- **terraform.tfvars** – Environment-specific values

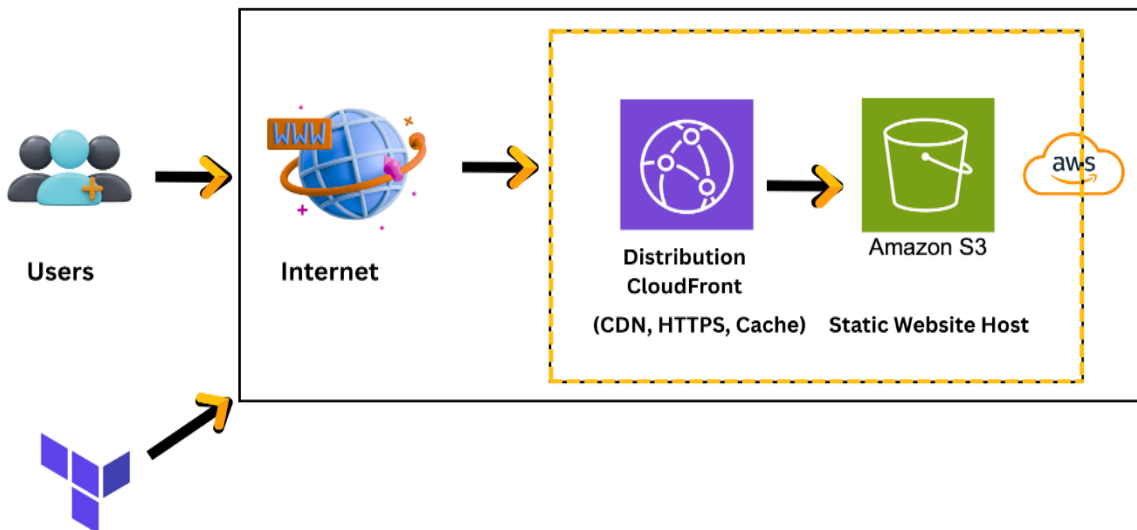
This modular structure improves clarity, maintainability, and scalability.

3. Architecture Interaction and Process Flow

The system interaction is designed as follows:

1. A user accesses the CloudFront domain via a web browser.
2. The request is routed to the nearest CloudFront edge location.
3. CloudFront checks whether the content is cached.
4. If not cached, CloudFront retrieves the content from the S3 bucket (origin).
5. The content is delivered to the user and cached for future requests.

Terraform automates the creation and configuration of all required AWS resources. After deployment, the website is accessed exclusively through the CloudFront domain rather than directly via S3.



4. Architectural Characteristics and Design Decisions

This architecture ensures scalability (automatic scaling of S3 and CloudFront), high availability (fully managed AWS services), performance optimization (edge caching), automation (IaC deployment), and cost efficiency (no compute resources such as EC2 required).

No EC2 or Auto Scaling group is used because the website is purely static and does not require backend processing. Serverless services reduce operational complexity while maintaining optimal performance and cost balance.